Unity Programmer Task – Rafael Melloni

A small 3D third-person prototype. The player picks up a pistol, ammo and medkits, manages them in a slot-based inventory and plays a shooting gallery: knock down every target before the 1:30 timer runs out, while missed shots hit the back wall and hurt you.

The Inventory system is split into three layers: Data, logic and UI.

Data: each item type is an ItemData ScriptableObject with its ID, name, description, icon and max stack size. Consumables and weapons inherit from it and override TryUse, so each item defines its own effect. ItemInstance represents a concrete stack and stores what changes at runtime: quantity and loaded ammo.

Logic: Inventory is a plain C# class that holds an array of slots. AddItem fills existing stacks first, then empty slots, and returns the amount that didn't fit. MoveItem merges compatible stacks or swaps their contents. UseItem calls TryUse and only consumes the item if it returns true. This layer has no reference to the UI.

UI: InventoryView subscribes to the inventory's OnChanged event and redraws every slot. Each slot handles drag and drop, a hover tooltip and a right-click menu to use, equip or drop items.

Save System: The game saves to a JSON file whenever the inventory, health or equipped weapon changes. Each item is stored by its ID and slot index, together with its quantity and loaded ammo, and an ItemDatabase restores the original assets when loading. It also saves the equipped weapon, the player's health and the items in the world.

Extras:Once the requirements were done, I added extras: aiming and hitscan shooting with reloading, a timed shooting gallery, an audio system, damage feedback and start, pause and game over menus.

Self-Assessment: I'm happy with how decoupled the inventory is and with how I managed my time, delivering every required system within the deadline while still having room for extras. With more time, I would create a weapon-specific instance type and add proper shooting and reload animations.